Ayudantía 4: Modelos predictivos de aprendizaje supervisado
IIC2115

Objetivo de la ayudantía

La idea es poder entender mejor el flujo completo de un problema de aprendizaje supervisado.

Algunas cosas a cubrir:

- ¿Qué es aprendizaje supervisado?
- ¿Qué representan X e y?
- ¿Por qué separar train y test?
- ¿Qué hacen `fit`, `predict` y `accuracy`?
- ¿Por qué a veces hay que preprocesar los datos?

¿Qué es aprendizaje supervisado?

Idea central

Aprendizaje supervisado es un enfoque en el que entrenamos un modelo usando ejemplos donde ya conocemos la respuesta correcta.

Ejemplo:

- Tenemos medidas de flores
- Queremos predecir su especie
- Cada ejemplo viene con su etiqueta real

En simple:

$$\text{datos de entrada} \rightarrow \text{modelo} \rightarrow \text{predicción}$$

Cómo se representan los datos

En `scikit-learn`, normalmente trabajamos con:

- X: variables de entrada o *features*
- y: variable objetivo o etiqueta

Interpretación:

- cada fila de X = un ejemplo
- cada columna de X = una característica
- cada valor de y = lo que queremos predecir

Ejemplo:

- X: largo del pétalo, ancho del pétalo, largo del sépalo, ancho del sépalo
- y: especie de la flor

Clasificación vs regresión

En aprendizaje supervisado, no todos los problemas son del mismo tipo.

Clasificación

La variable objetivo y es una **categoría** o clase.

- ejemplos: especie de flor, spam/no spam, aprobado/reprobado
- la predicción pertenece a un conjunto finito de etiquetas

Regresión

La variable objetivo y es un **valor numérico continuo**.

- ejemplos: precio de una casa, temperatura, ingreso mensual
- la predicción es un número

Idea clave: antes de entrenar, hay que identificar correctamente qué tipo de problema estamos resolviendo.

Pipeline general

$$\text{Datos} \rightarrow \text{Preparación} \rightarrow \text{Split} \rightarrow \text{Entrenamiento} \rightarrow \text{Predicción} \rightarrow \text{Evaluación}$$

Pasos típicos:

1. elegir variables de entrada y objetivo
2. limpiar / transformar datos
3. separar train y test
4. entrenar el modelo
5. predecir sobre datos no vistos
6. medir desempeño

¿Por qué separar train y test?

Idea clave

Un modelo no debe evaluarse con los mismos datos que usó para aprender.

Porque si no:

- puede memorizar patrones del conjunto de entrenamiento
- parecer muy bueno artificialmente
- fallar cuando recibe datos nuevos

Entonces:

- `train`: para aprender
- `test`: para comprobar si generaliza

Ejemplo de split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Lectura de este código:

- 80 % de los datos para entrenamiento
- 20 % para prueba
- `random_state` fija la semilla para reproducibilidad (para que cuando corran ustedes el algoritmo, no cambie a cada rato)

Preprocesamiento

No siempre los datos están listos para entrenar un modelo.

Algunas tareas comunes:

- convertir variables categóricas a números
- escalar variables numéricas
- manejar valores faltantes
- seleccionar columnas útiles

Importante

Preprocesar no es 'adorno': muchas veces es parte esencial del desempeño del modelo.

Ejemplo de escalamiento

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Ojo con esto:

- `fit_transform` se aplica al train
- `transform` se aplica al test
- no se debe "aprender" del test

¿Qué es data leakage?

Idea central

Hay **data leakage** cuando, durante el entrenamiento o preprocesamiento, el modelo recibe información que en un escenario real no debería conocer.

Consecuencia:

- el modelo parece rendir muy bien
- pero ese rendimiento es engañoso
- al usar datos nuevos, el desempeño real cae

Ejemplos típicos:

- escalar usando todo el dataset antes del split
- imputar valores faltantes usando train y test juntos
- incluir en X una variable que revela directa o indirectamente el valor de y

Regla práctica

Todo lo que aprende de los datos debe ajustarse solo con `train` y luego aplicarse a `test`.

Entrenar y predecir

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Interpretación:

- `fit`: el modelo aprende a partir de los datos de entrenamiento
- `predict`: genera predicciones para datos no vistos

Evaluación básica

```
from sklearn.metrics import accuracy_score

acc = accuracy_score(y_test, y_pred)
print(acc)
```

Accuracy:

$$\text{accuracy} = \frac{\text{número de predicciones correctas}}{\text{número total de predicciones}}$$

Ejemplo: si el modelo acierta 18 de 20 ejemplos, entonces

$$\text{accuracy} = \frac{18}{20} = 0{,}9$$

Comparar modelos

Una misma tarea puede resolverse con distintos modelos:

- KNN
- Árbol de decisión
- SVM

Idea importante: aunque cambie el algoritmo, el pipeline general sigue siendo parecido:

$$\texttt{fit} \rightarrow \texttt{predict} \rightarrow \text{evaluar}$$

Eso hace que `scikit-learn` sea cómodo para experimentar y comparar.

Por qué esta diferencia importa

Clasificación y regresión requieren decisiones distintas en todo el pipeline.

- **Modelo:** no siempre existe una única versión
  - ejemplo: `DecisionTreeClassifier` vs `DecisionTreeRegressor`
  - ejemplo: `KNeighborsClassifier` vs `KNeighborsRegressor`
- **Salida esperada:**
  - clasificación $\rightarrow$ clase o etiqueta
  - regresión $\rightarrow$ valor numérico
- **Evaluación:**
  - clasificación $\rightarrow$ accuracy, precision, recall, F1, etc.
  - regresión $\rightarrow$ MAE, MSE, RMSE, R^2, etc.

Advertencia

No tiene sentido evaluar una regresión con `accuracy`, porque en regresión no estamos prediciendo clases.

Dudas frecuentes

1. ¿Por qué una columna va en y y no en X?

Porque y es justamente lo que queremos predecir; X contiene la información disponible para hacerlo.

2. ¿Por qué no evaluamos con train?

Porque eso no mide generalización; solo muestra qué tan bien el modelo recuerda los datos que ya vio.

Dudas frecuentes

3. ¿Por qué escalar?

Porque algunos modelos son sensibles a la magnitud de las variables y pueden verse afectados si una columna domina a otra por su escala.

4. ¿Qué significa que un modelo "aprende"?

Significa que ajusta parámetros internos para capturar patrones entre X e y.

Dudas frecuentes

5. ¿Qué pasa si el accuracy es alto?

No siempre significa que el modelo sea excelente: depende del contexto, del dataset y de si la métrica elegida es adecuada.

6. ¿Siempre hay que usar el modelo más complejo?

No. Muchas veces un modelo simple, bien entendido y bien evaluado, es una mejor elección.

Dudas frecuentes

7. ¿Cómo sé si mi problema es clasificación o regresión?

Depende de la variable objetivo y: si es una categoría, es clasificación; si es un valor numérico continuo, es regresión.

8. ¿Puedo usar cualquier métrica con cualquier modelo?

No. La métrica debe ser coherente con el tipo de problema. Por ejemplo, `accuracy` sirve para clasificación, no para regresión.

Errores comunes

- confundir `features` con `target`
- entrenar con columnas que no corresponden
- hacer `fit` también sobre test en preprocesamiento
- interpretar mal el accuracy
- comparar modelos sin mantener el mismo criterio de evaluación

Ideas clave para recordar

1. Aprendizaje supervisado usa ejemplos con respuesta conocida.
2. X son las entradas; y es la salida que queremos predecir.
3. Train sirve para aprender; test sirve para evaluar.
4. Preprocesar datos suele ser parte fundamental del trabajo.
5. El objetivo no es solo entrenar un modelo, sino entender el flujo completo.

Vamos a los ejercicios